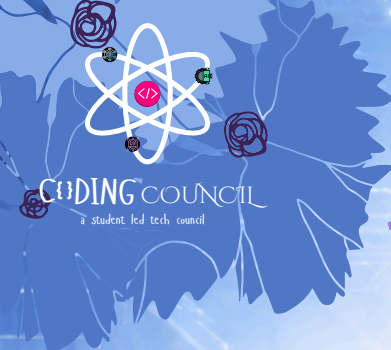




PYTHON WINTER SPRINT 2026



Core Problem of 11th Jan 2026 : Banking System

Each Day have one constant theme and three different tracks : Foundation Track especially due to W3B Collaboration, Core Track for those following since DAY-1 and the Project Track for those who have their basics of Python Cleared, they will make these end-to-end projects

FOUNDATION TRACK (FOR ABSOLUTE BEGINNERS AND NEW JOINERS)

[Expected Time to Finish – 2.5hrs]

Resource: <https://www.youtube.com/watch?v=UrsmFxEIp5k> from 4hrs 50 minutes to 6hrs 14 minutes

Core Topic: Functions, Recursions and File I/O

🌱 Problem Statement

Write a Python program that:

- Asks the user for:
 - Account holder name
 - Initial balance
- Saves account details to a file bank.txt
- Uses a recursive function to allow the user to:
 - Deposit money
 - Withdraw money
 - Exit
- After exit:
 - Read the file
 - Display final account summary

📌 Instructions (Simple Language)

- Use functions for deposit, withdraw, and display
- Use recursion instead of loops to repeat banking operations
- Use file handling to store balance updates
- Keep logic very simple

🧠 Expected Flow

- Program starts
- Account is created and saved
- Recursive menu runs:
 - Deposit
 - Withdraw
 - Exit
- Final balance is shown from file

SAMPLE RUN:

```
Enter account holder name: Ayaan
Enter initial balance: 5000
```

```
Choose operation:
```

- ```
1. Deposit
2. Withdraw
3. Exit
```

```
Enter choice: 1
```

```
Enter deposit amount: 2000
Amount deposited successfully.
```

```
Choose operation:
```

- ```
1. Deposit
2. Withdraw
3. Exit
```

```
Enter choice: 2
```

```
Enter withdrawal amount: 1500
Withdrawal successful.
```

```
Choose operation:
```

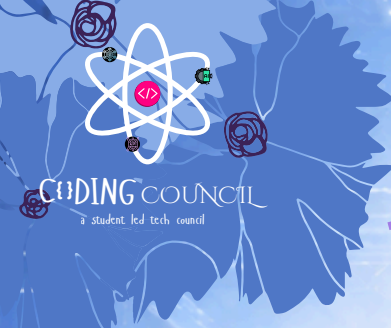
- ```
1. Deposit
2. Withdraw
3. Exit
```

```
Enter choice: 3
```

```
--- ACCOUNT SUMMARY ---
```

```
Account Holder: Ayaan
Final Balance: ₹5500
```

Beginners Please ask in Group if you get to encounter any problem, any kind of problem in this, or just DM me  
Every One was once a beginner !!



# PYTHON WINTER SPRINT 2026



## # CORE TRACK (FOR STUDENTS WHO HAVE BEEN LEARNING FROM DAY 1)

Core Topic: Object Oriented Programming

Resource: <https://www.youtube.com/watch?v=UrsmfxEIp5k> from 7hr 23 minutes till 9hr 16 minutes

[Expected Time to Finish - 3hrs]

### ● Core Track - Banking System (Inheritance Focused)

Simple Banking System using  
Inheritance

#### 🎯 Objective

Understand inheritance by creating a  
base account class and extending it  
for specific account types.

#### 🧠 Concept Being Tested

- Base class (BankAccount)
- Child class (SavingsAccount)
- Method reuse and override
- super() usage

#### ✳ Problem Statement

Write a Python program that:

- Creates a base class BankAccount with:
  - account\_holder
  - balance
  - methods:
    - deposit(amount)
    - show\_balance()
- Creates a child class SavingsAccount that:
  - Inherits from BankAccount
  - Adds a withdraw(amount) method
  - Prevents withdrawal if balance is insufficient
- Create an object of SavingsAccount and perform:
  - Deposit
  - Withdraw
  - Balance check

Sample Run:

Enter account holder name: Arshad

Enter initial balance: 5000

Depositing 2000...

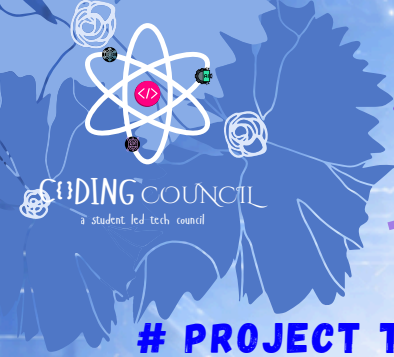
Deposit successful.

Withdrawing 3000...

Withdrawal successful.

Current Balance: ₹4000





# PYTHON WINTER SPRINT 2026



## # PROJECT TRACK (FOR ADVANCED PARTICIPANTS, MINI PROJECT)

[Expected Time to Finish – 3-4 hours]

### 🎯 Objective

Build a CLI-based Banking System that supports:

- User authentication
- Account management
- Transactions with rules
- Audit logs
- Persistent storage

Think of this as a mini core banking engine.

### 🧩 Mandatory Features (Must-Have)

#### 🔑 User Authentication System

- Register user with:
  - username
  - password
- Login required before any operation
- Store credentials in users.json

#### 🏠 Account Management

Each user has:

- Account number (auto-generated)
- Account type (Savings / Current)
- Balance

Data stored in accounts.json

#### 💰 Transaction Engine

Support:

- Deposit
- Withdraw
- Balance inquiry

Rules:

- No negative deposits
- Cannot withdraw more than balance

Minimum balance rule for Savings account

## # PROJECT TRACK (FOR ADVANCED, SYSTEM-LEVEL PROBLEM SOLVING)

### Audit & Logging System (IMPORTANT)

Every transaction must:

- Generate a unique transaction ID
- Store:
  - account number
  - transaction type
  - amount
  - timestamp
  - status (SUCCESS / FAILED)

Stored in transactions.json

### Business Rules

Implement at least 3 rules, for example:

- Withdraw > ₹50,000 → flag transaction
- More than 3 failed withdrawals → warning
- Zero balance → account freeze warning

## # PROJECT TRACK – SAMPLE RUN

```
=====
SECURE BANKING SYSTEM
=====
```

1. Register

2. Login

Choose option: 1

Enter username: arshad

Enter password: 1234

Account created successfully.

Account Number: 100231

```
=====
1. Login
```

2. Exit

Choose option: 1

Username: arshad

Password: 1234

Login successful.

```

Choose Operation:
```

1. Deposit

2. Withdraw

3. Check Balance

4. View Transaction History

5. Logout

```

Enter choice: 1
```

Enter amount to deposit: 20000

Transaction Successful.

Transaction ID: TXN102938

Current Balance: ₹20000

```

Enter choice: 2
```

Enter amount to withdraw: 60000

Transaction Failed: Insufficient balance.

Transaction ID: TXN102945

```

Enter choice: 3
```

Current Balance: ₹20000

Enter choice: 4

```
----- TRANSACTION HISTORY -----
```

TXN102938 | Deposit | ₹20000 | SUCCESS

TXN102945 | Withdraw | ₹60000 | FAILED

```

Enter choice: 5
```

Logging out...

Thank you for using Secure Banking System.